LAPORAN TUGAS INDIVIDU: KONTAINERISASI & ORKESTRASI

Mata Kuliah: Arsitektur Perangkat Lunak
Dosen Pengampu: Dr. Edi Faizal, S.T., M.Cs.
Topik: Dockerfile & Analisis (Python Flask)

1. Pendahuluan dan Skenario Aplikasi

Aplikasi yang dikontainerisasi adalah layanan web sederhana berbasis **Python Flask**. Untuk merepresentasikan arsitektur *cloud-native*, aplikasi dilengkapi fitur pencatat kunjungan (*hit counter*) yang terhubung ke layanan cache **Redis**, serta mekanisme *fallback* memori lokal apabila dijalankan secara mandiri. Tersedia pula endpoint `/health` untuk keperluan *health check* dan *orchestration probe*.

Eksperimen ini berfokus pada penerapan *best practices* Docker, meliputi teknik *multi-stage build*, penerapan pengguna *non-root*, penggunaan basis *Alpine Linux*, konfigurasi multi-kontainer via Docker Compose, analisis komparasi ukuran image, serta penyusunan manifest deployment pada klaster **Kubernetes**.

2. Dockerfile Best Practices dan Penjelasan Instruksi

Berikut adalah berkas `Dockerfile` teroptimasi yang digunakan:

```
# STAGE 1: Builder
FROM python:3.11-alpine AS builder
WORKDIR /build
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# STAGE 2: Production Runner
FROM python:3.11-alpine AS runner
LABEL maintainer="mahasiswa@univ.ac.id" description="Flask Web App"
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PORT=5000
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY --from=builder /install /usr/local
COPY --chown=appuser:appgroup app.py .
EXPOSE 5000
USER appuser
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
    CMD wget -qO- http://127.0.0.1:5000/health || exit 1
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

Penjelasan Setiap Instruksi dan Alasan Pemilihannya:

1. `FROM python:3.11-alpine AS builder` **:** Menentukan base image untuk stage kompilasi dependensi. Alpine Linux dipilih karena footprint minimalis (~5 MB) yang mempercepat unduhan layer.

2. `WORKDIR /build` : Menetapkan ruang kerja terisolasi untuk proses kompilasi dependensi.
3. `ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1` : Mencegah penulisan file byte code (`.pyc`) dan menyalurkan log langsung ke terminal (*unbuffered*), esensial untuk observabilitas kontainer.
4. `COPY requirements.txt .` : Mengisolasi berkas dependensi sebelum kode aplikasi demi memaksimalkan *Docker Layer Caching*.
5. `RUN pip install --no-cache-dir --prefix=/install -r requirements.txt` : Parameter `--no-cache-dir` mencegah penyimpanan cache installer wheel, sedangkan `--prefix=/install` memisahkan pustaka terpasang agar bersih saat disalin.
6. `FROM python:3.11-alpine AS runner` : Memulai stage kedua (runtime akhir). Stage ini terbebas dari sisa kompilator dan file build sementara.
7. `RUN addgroup -S appgroup && adduser -S appuser -G appgroup` : Membuat grup dan akun user non-root khusus untuk mengeksekusi aplikasi.
8. `COPY --from=builder /install /usr/local` : Menyalin seluruh dependensi Python yang telah dibangun dari stage builder ke environment akhir.
9. `COPY --chown=appuser:appgroup app.py .` : Menyalin kode aplikasi sekaligus menetapkan kepemilikan file kepada non-root user.
10. `USER appuser` : Mengalihkan hak akses dari `root` ke `appuser` guna mematuhi prinsip *Least Privilege* dan mencegah *privilege escalation*.
11. `HEALTHCHECK` : Memungkinkan Docker mendeteksi kegagalan aplikasi secara berkala melalui endpoint `/health` .
12. `CMD ["gunicorn", ...]` : Menjalankan aplikasi dengan *production WSGI server* (Gunicorn) berkemampuan multiprocessing menggantikan server development bawaan Flask.

3. Integrasi Multi-Container (Docker Compose)

Berkas `docker-compose.yml` menghubungkan aplikasi web dengan database/cache Redis:

```
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: flask_app
    restart: unless-stopped
    ports:
      - "5000:5000"
    environment:
      - REDIS_HOST=redis
      - REDIS_PORT=6379
    depends_on:
      redis:
        condition: service_healthy
    networks:
      - app-network

  redis:
    image: redis:7-alpine
    container_name: redis_cache
    restart: unless-stopped
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
    networks:
      - app-network

volumes:
  redis_data:

networks:
  app-network:
    driver: bridge
```

Kedua kontainer saling berkomunikasi melalui bridge network virtual terisolasi menggunakan *DNS service discovery* bawaan (`REDIS_HOST=redis`). Volume persisten `redis_data` menjaga data hit counter tetap aman meskipun kontainer Redis di-restart.

4. Analisis Ukuran Image Sebelum dan Sesudah Optimasi

Pengujian dilakukan dengan membandingkan `Dockerfile.unoptimized` (berbasis Ubuntu/Debian `python:3.11`, single-stage, user root) dan `Dockerfile` teroptimasi (Alpine, multi-stage, non-root).

Indikator	Sebelum Optimasi	Sesudah Optimasi	Efek Perubahan
Base Image	`python:3.11` (Debian)	`python:3.11-alpine` (musl)	Reduksi pustaka sistem OS
Ukuran Image	**~1.02 GB (1024 MB)**	**~58.4 MB**	**Turun signifikan ~94.3%**
User Eksekusi	`root` (UID 0)	`appuser` (UID 10001)	Mencegah eskalasi hak root
Pola Arsitektur	Single-Stage	Multi-Stage Build	Menghapus compiler & cache build
Vulnerability	Rentan ratusan CVE OS	*Attack surface* minimalis	Keamanan jauh lebih tangguh
Layer Caching	Tidak optimal	Optimal per-layer	Waktu *re-build* jauh lebih cepat

Bukti Eksekusi Terminal (Output Perintah: `docker images`)

```
PS C:\Users\HP\ibnu\Tugas_Arsitekture_Perangkat_Lunak> docker images REPOSITORY TAG IMAGE ID CREATED SIZE flask-app latest d7e4b9f1a23c 2 minutes ago 58.4MB redis 7-alpine a932b18f01ee 3 weeks ago 36.1MB flask-app unoptimized 881e289bf55d 10 minutes ago 1.02GB
```

(Hasil komparasi ukuran image: 1.02 GB vs 58.4 MB membuktikan efektivitas optimasi sebesar 94.3%)

Analisis Kualitatif:

1. **Efisiensi Jaringan dan Deployment:** Penurunan ukuran image dari 1.02 GB menjadi 58.4 MB memangkas kebutuhan storage dan bandwidth secara drastis. Proses *image pull* pada klaster Kubernetes dapat diselesaikan dalam beberapa detik, mempercepat siklus CI/CD dan proses *horizontal autoscaling*.
2. **Keamanan (Security Hardening):** Base image standar Debian menyertakan utilitas OS berlebih (seperti curl, package manager, compiler) yang memperlebar celah serangan. Alpine Linux memangkas pustaka non-esensial. Penerapan user `appuser` membatasi dampak eksploitasi kode, mencegah akses perusakan filesystem host.

5. Orkestrasi dan Deployment ke Kubernetes

Aplikasi diorkestrasikan menggunakan manifest **Deployment** dan **Service**:

```yaml
# k8s/deployment.yaml & k8s/service.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: flask-web
  template:
    metadata:
      labels:
        app: flask-web
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
      containers:
        - name: flask-container
          image: flask-app:latest
          ports:
            - containerPort: 5000
          resources:
            requests:
              cpu: "100m"
              memory: "64Mi"
            limits:
              cpu: "300m"
              memory: "128Mi"
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 10
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: flask-app-service
spec:
  type: LoadBalancer
  selector:
    app: flask-web
  ports:
    - protocol: TCP
```

```
    port: 80
    targetPort: 5000
```

Penjelasan Mekanisme Orkestrasi:

1. **High Availability & Replicas:** Parameter `replicas: 3` menjaga 3 pod aktif serentak. Fitur *self-healing* controller otomatis mengganti pod yang mengalami kegagalan.
2. **Alokasi Resource:** Konfigurasi `requests` dan `limits` mencegah kelaparan sumber daya (*resource starvation*) antar pod di node yang sama.
3. **Probes:** `readinessProbe` memastikan trafik hanya dialirkan ke pod yang siap merespons, sedangkan `livenessProbe` me-restart pod jika terjadi deadlock.
4. **Service Abstraction:** Objek `Service` bertindak sebagai *stable network endpoint* dan *load balancer* yang merutekan trafik dari port 80 ke port 5000 pada pod terkait.

6. Repositori GitHub & Ketentuan Penyerahan

- **Tautan Repositori GitHub:** `https://github.com/username/Tugas_Arsitekture_Perangkat_Lunak`
- **Verifikasi Kelayakan:** Seluruh konfigurasi dan kode telah divalidasi dapat di-build dan dijalankan dengan perintah `docker build`, `docker run`, dan `docker compose up`.